K U S I N A K O D E

Kusina Kode System Primer

A lecture-style study module for the capstone defense. Every concept is taught the same way, and every example points to code that actually runs in Kusina Kode: the Android app, the PHP API on Azure, and the admin console.

1
Definition
The textbook concept.

2
Why it matters here
What problem it solves in Kusina Kode.

3
How we applied it
The real file, class or table.

4
In plain words
How to say it to a non-programmer.

5
Insights
Why we chose it, and supporting evidence.

BEFORE YOU START

Study plan

Six sessions of about 45 minutes. Read a unit, then answer its "Panel might ask" lines out loud without looking. Units 1 to 3 are the program itself; units 4 and 5 are how it is protected and hosted, and unit 6 ties it all together in the diagrams.

SESSION	UNIT	GOAL	YOU CAN EXPLAIN…
1	OOP & architecture	Know how the code is organised	Why the app is split into screens, ViewModels, rules and repositories
2	Data structures	Know where data lives and why	Why a map, a set or a chain was the right container
3	Algorithms	Know the core logic step by step	How a guess becomes coloured tiles, and how XP and ranks are computed
4	Cybersecurity	Know what protects players and rewards	Passwords, tokens, lockout, encryption, and why the ledger can't be faked
5	Cloud computing	Know where it runs and how well	Each Azure service, what it costs us, and what the stress test showed
6	System diagrams	Walk through each figure in a minute	The cloud, security and system architecture, the use cases and the data flows

UNIT 1

Object-oriented programming & architecture

The app is written in Kotlin and the admin console in C#; both are object-oriented. In the app alone there are 114 data classes, 15 interfaces, 16 enums, 59 singleton objects and 8 ViewModels.

1.1 Classes and objects `data class · enum · object`

DEFINITION

A **class** is a blueprint that bundles data (properties) with the behaviour that works on it (methods). An **object** is one instance made from that blueprint.

WHY IT MATTERS HERE

Kusina Kode handles many "things" with their own data: levels, players, badges, ingredients, attempts. Modelling each as a class keeps its data and rules together and consistent everywhere they are used.

HOW WE APPLIED IT

- **Data classes** for records, e.g. `PlayerProgress` (points, streaks, solved levels) and `LevelData`. Kotlin generates equality and copying for them, so `current.copy(totalPoints = …)` makes a new updated record safely.
- **Enums** for fixed choices: `Region` (Luzon, Visayas, Mindanao), `Rarity`, `TileState` (Correct, SemiCorrect, Wrong), `PowerUp`.
- **Singleton objects** for one shared rulebook: `WordleEngine`, `ScoreRules`, `ChefRank`, `IngredientCatalog`.

IN PLAIN WORDS

A class is the recipe card; an object is the dish you cook from it. Every level in the game is cooked from the same "level" recipe, so they all have a word, a region and ingredients.

INSIGHTS

Data classes are **immutable by habit** in the app: progress is replaced with `copy()` rather than edited in place, which avoids half-updated scores when several screens read the same data.

Enums stop impossible values: a tile can only be one of three states, so a typo like "corect" can't compile.

Panel might ask: "Why use a singleton for the scoring rules?" Because there must be exactly one rulebook; two copies could score the same round differently.

1.2 Encapsulation `private · read-only views`

DEFINITION

Hiding an object's internal state and exposing only controlled ways to read or change it.

WHY IT MATTERS HERE

Game state (the board, the timer, the KK balance) must only change through the game's rules; a screen should never be able to set "you won" directly.

HOW WE APPLIED IT

```
// GameViewModel.kt
private val _uiState = MutableStateFlow(…)   // only the ViewModel can change it
val uiState = _uiState.asStateFlow()         // screens get a read-only view
```

- The same pattern guards `PantryViewModel`, `KkBalance` and the gamification store (`private set` on `gamification` in `KusinaKodeApp`).
- On the web, ASP.NET pages reach data only through the composition root and service classes (`DishService`, `IngredientService`), never straight to SQL.

IN PLAIN WORDS

The kitchen has one door for orders. Diners can see the food, but only the cook can change it.

INSIGHTS

This is also a security property: if the screen can't change the score, a bug in a screen can't award free KK. The server re-checks everything anyway (see 4.2).

1.3 Abstraction `interfaces`

DEFINITION

Describing *what* something does through an interface while hiding *how* it does it.

WHY IT MATTERS HERE

The game needs data from several places: the phone's storage, the PHP API, the chain. The game logic shouldn't care which one answered.

HOW WE APPLIED IT

- 15 repository interfaces, e.g. `GamificationRepository`, `PantryRepository`, `LevelRepository`, `AuthRepository`, `WalletRepository`.
- The coordinator only knows the interface: `GamificationCoordinator(repository: GamificationRepository, …)`.
- The web console does the same with `IIngredientStore`, `IReelStore` and `IAdminSignIn` in `Web_KusinaKode/Abstractions`.

IN PLAIN WORDS

> The game asks "give me the pantry". It doesn't need to know whether the answer came from the phone or from the server.

INSIGHTS

Abstraction is what made the unit tests possible: tests swap in fake repositories, so `ProgressRules` and `BadgeRules` are tested without a phone or internet.

It also let the team move from local XAMPP to Azure without rewriting the game logic.

1.4 Inheritance `extends Android & ASP.NET base classes`

DEFINITION

A class reusing and extending another class's behaviour (a subclass "is a" kind of its parent).

WHY IT MATTERS HERE

Android and ASP.NET already provide lifecycle, memory and request handling. Inheriting from their base classes gives Kusina Kode those features for free.

HOW WE APPLIED IT

- `class KusinaKodeApp : Application()` : runs once when the app starts (sets up the network status, gamification store, chain coordinator).
- `class GameViewModel(…) : ViewModel()` and seven other ViewModels: survive screen rotation and are cleared automatically.
- Each admin page (`Levels.aspx.cs` , `Audit.aspx.cs` …) inherits from ASP.NET's `Page` ; the shared `Admin.master` layout gives every page the sidebar and sign-in check.

IN PLAIN WORDS

Instead of building our own kitchen, we rented one and added our own recipes.

INSIGHTS

The code favours **composition over deep inheritance**: our own classes rarely extend each other. They combine small parts (a repository, a coordinator) instead, which keeps them easier to test and change.

1.5 Polymorphism `one interface, many forms`

DEFINITION

One interface or type can stand for many concrete forms, and the right behaviour is chosen at run time.

WHY IT MATTERS HERE

Some features have several sources or outcomes: the same "progress" can be stored locally or remotely, and a guess can be valid or invalid in several ways.

HOW WE APPLIED IT

- **Interface polymorphism:** `LocalGamificationRepository`, `RemotePantryRepository`, `RemoteAttemptRepository` are concrete forms of their repository interfaces.
- **Sealed types:** `WordleEngine.GuessValidation` is `Valid`, `Incomplete`, `WrongLength(expected, actual)` or `NotLetters`. The compiler forces every screen to handle all four.
- `NotificationGlyph` is either `Art` (a bundled picture) or `Remote` (a web picture), drawn differently.

IN PLAIN WORDS

"Serve the dish" works for adobo or sinigang; each one knows how it should be plated.

INSIGHTS

Sealed types turn "did we handle every case?" into a compile error instead of a crash. When a new validation case is added, every place that forgot it stops compiling.

1.6 MVVM, Repository & Clean Architecture `design patterns`

DEFINITION

MVVM separates the View (screen), ViewModel (state and actions) and Model (data and rules). The **Repository** pattern hides data sources behind one class. **Clean Architecture** arranges code in layers where inner rules never depend on outer details. The **Observer** pattern lets screens react automatically when data changes.

WHY IT MATTERS HERE

The project rule (`CLAUDE.md`) is that screens never call the network or database directly. Four people work on different modules, so clear layers prevent stepping on each other.

HOW WE APPLIED IT

- **App:** Compose screens → ViewModels exposing `StateFlow` (observer) → repositories → `KusinaApi` (Ktor HTTP client). Pure rules live in `domain/` with no Android code.
- **Web console:** Kyla split it into `KusinaKode.Domain` (rules, with tests), `Application` (services), `Data` (stores) and `Presentation` (pages), wired in one composition root.

IN PLAIN WORDS

> The waiter (screen) takes the order, the head cook (ViewModel) decides what happens, and the pantry staff (repository) fetch ingredients. Nobody does someone else's job.

INSIGHTS

Because the rules are separate, the same tile-scoring and XP logic is unit-tested on its own (`WordleEngineTest` , `ScoreRulesTest` , `ProgressRulesTest`). Defect D-01 (wrong colours for repeated letters) was found and fixed in that layer.

Panel might ask: "Why not let screens call the API?" A slow network would freeze the screen, and every screen would duplicate error handling. The ViewModel layer does both in one place.

UNIT 2

Data structures

Choosing the right container decides how fast the app can find, check and store things. Each structure below is used on purpose.

2.1 Lists and arrays `the guess grid`

DEFINITION

An ordered collection where each element has a position (index 0, 1, 2…).

WHY IT MATTERS HERE

A guess is a row of letters, and each letter's colour depends on its **position**. Order is the whole point.

HOW WE APPLIED IT

`WordleEngine.evaluate()` returns `List<TileState>`, one state per letter, built with `MutableList(ans.length) { TileState.Wrong }`. The board is a list of these rows: six rows for every dish (`MAX_ATTEMPTS = 6`).

IN PLAIN WORDS

A row of numbered boxes. Box 1 checks letter 1, box 2 checks letter 2, and so on.

INSIGHTS

The list is sized from the answer's length, not fixed. That is how the project meets its **variable word length** requirement: ADOBO makes 5 boxes, SINIGANG makes 8, with no special cases.

2.2 Hash maps `key → value in one step`

DEFINITION

A map stores key-value pairs and finds a value from its key in roughly constant time, O(1), no matter how many entries it holds.

WHY IT MATTERS HERE

The app constantly looks things up by name or id: a letter's remaining count, an ingredient's picture, an ingredient's details.

HOW WE APPLIED IT

- `remaining = mutableMapOf<Char, Int>()` in the tile algorithm counts unused answer letters.
- `IngredientArt.byName` maps about 200 dataset ingredient names to their photos.
- `IngredientCatalog.byId` maps ids like `ing_tuna_fresh` to ingredient records.

IN PLAIN WORDS

A labelled spice rack: you go straight to "garlic" instead of opening every jar.

INSIGHTS

Scanning a list of 123 ingredients for every tile on a dish page would repeat that work for each tile; a map answers each lookup at once. Maps also made the 1.23 fix simple: when a web name isn't in the picture map, the app looks it up by id instead.

2.3 Sets `membership without duplicates`

DEFINITION

A collection of unique items, built for fast "is X in here?" checks.

WHY IT MATTERS HERE

"Has this player already solved this dish?" decides whether a win pays XP and KK. It must be fast and must never count a dish twice.

HOW WE APPLIED IT

`PlayerProgress.solvedLevels` is a `Set<Int>`. In `ProgressRules.advance()`: `if (event.levelId in current.solvedLevels) return current`, so a replay changes nothing. `regionsSolved` and `newIngredientIds` are sets too.

IN PLAIN WORDS

A stamp card: each dish gets one stamp, and stamping it again doesn't add a second.

INSIGHTS

This set is the anti-farming rule. Defect D-05 ("replaying a solved dish awarded points") was fixed by checking this set first. The stress test confirmed it: replays showed "Already solved, +0".

2.4 Streams and queues `StateFlow · event bus · chain queue`

DEFINITION

A **queue** processes items in arrival order (first in, first out). A **stream** (flow) delivers a sequence of values over time to whoever is listening.

WHY IT MATTERS HERE

Many things happen after a win: XP, badges, KK, the chain receipt, the palayok. They must happen in order, even if the player is already on another screen.

HOW WE APPLIED IT

- Screens observe `StateFlow` streams; when the value changes, the screen redraws.
- `GameEvents` / `GamificationEvents` broadcast "round completed", and `GamificationCoordinator` listens for the whole app lifetime.
- `ChainQueue` lines up reward transactions for the ledger; on the server, a job settles anything still pending every 15 minutes.

IN PLAIN WORDS

Orders go on a rail in the kitchen and are cooked one after another, so none are lost when it gets busy.

INSIGHTS

Defect D-31 ("two rewards at once both fail") is exactly the problem queues solve: two writes racing for the same ledger. Ordering them, plus the server-side settle job, removes the race.

2.5 Hash-linked chain `blocks.jsonl`

DEFINITION

A linked list where each element (block) stores the **hash of the previous block**, so the whole list is chained together. This is the core data structure of a blockchain.

WHY IT MATTERS HERE

The study's third objective is a tamper-resistant reward record. Every KK earned or spent must be provable later.

HOW WE APPLIED IT

The server keeps the chain in `/var/lib/kusinakode/chain/blocks.jsonl`, one block per line, with each transaction in `txs/`. `lib/chain.php` computes each block's hash and checks the chain's tip; the database stores only receipts (`tx_ref` in `reward_claims`, `shop_unlocks`, `ingredient_draws`).

IN PLAIN WORDS

A receipt book where each page carries a fingerprint of the page before it. Tear out or change one page and every page after it stops matching.

INSIGHTS

Why a file and not a SQL table? Because the database is meant to be the editable working copy, while the chain is the evidence. An admin can edit a table, but cannot silently edit a block without breaking every later hash. The hourly integrity check on the Audit page detects exactly that.

2.6 Relational tables `20 tables · keys · relationships`

DEFINITION

Data stored in tables of rows and columns, linked by **primary keys** (unique id per row) and **foreign keys** (a column pointing at another table's key).

WHY IT MATTERS HERE

Players, levels, attempts, rewards and ingredients all relate to each other, and the admin console, analytics and leaderboard query across them.

HOW WE APPLIED IT

Azure SQL holds 20 tables (see the ERD). Examples: `attempts.user_id → users.id`, `attempts.level_id → levels.id`, enforced with `ON DELETE CASCADE`. Composite keys stop duplicates, e.g. `reward_claims`' key is `(user_id, kind, claim_key)`, so one first-dish bonus per player per dish.

IN PLAIN WORDS

Linked spreadsheets: the attempts sheet doesn't copy player names, it just points to the player's row.

INSIGHTS

The composite key on `reward_claims` makes double payouts impossible at the database level, not just in code. Cascading deletes explain a real event: deleting a level also deleted its attempts, which is why the console now refuses to remove played levels.

UNIT 3

Algorithms

The step-by-step procedures behind the game. Each one is small, deterministic and unit-tested.

3.1 Two-pass tile scoring `WordleEngine.evaluate · O(n)`

DEFINITION

A **multi-pass algorithm** walks the data more than once, each pass doing one job, so later passes can rely on what earlier passes decided.

WHY IT MATTERS HERE

This is the heart of the puzzle: turning a guess into green (correct), yellow (in the word, wrong place) and grey (not in the word), and it must handle repeated letters correctly.

HOW WE APPLIED IT

```
Pass 1: for each position i
          if guess[i] == answer[i] → Correct (green)
          else count answer[i] in remaining
Pass 2: for each position not yet green
          if remaining[guess[i]] > 0 → SemiCorrect (yellow),
remaining -1
          else → Wrong (grey)
```

Example: answer **SISIG**, guess **SINGS** → S I green, N grey, G yellow, the second S yellow (exactly what the booth screenshot shows).

IN PLAIN WORDS

First mark every letter that's in the exact right spot. Then, with the letters left over, mark the ones that exist somewhere else, and never more times than they appear in the answer.

INSIGHTS

The original Capstone 1 code used a one-pass `indexOf()`, which matched the first occurrence and showed too many yellows for repeated letters (defect D-01). Two passes plus a count map fix it.

Time is **O(n)** for an n-letter word: two passes, each with constant-time map lookups. That's instant even for long dish names.

Panel might ask: "Why two passes?" Because greens must be claimed first; otherwise a yellow could use up a letter that a later position needs for a green.

3.2 The XP formula `ScoreRules.pointsFor`

DEFINITION

A **deterministic scoring function**: the same inputs always give the same output, with no randomness.

WHY IT MATTERS HERE

XP must reward skill fairly and be reproducible: the same round always scores the same on any phone, which the acceptance criteria call "consistent across sessions".

HOW WE APPLIED IT

```
XP = 100                          // solved the dish
   + 20 × (6 - tries used)        // tries left over
   + 50 if solved on the 1st try
   + 60 (<30 s) / 40 (<60 s) / 20 (<120 s) / 0
```

Range: **100** (last try, slow) to **310** (first try, under 30 s). Losses and replays score 0.

IN PLAIN WORDS

You get 100 for solving it, plus bonuses for needing fewer tries, nailing it first time, and being quick.

INSIGHTS

Bonuses use **buckets** (under 30 s, under 60 s…) rather than exact seconds, which the code comments say makes a score "easy to explain to a player, and easy to re-derive if the chain ever needs to audit it".

3.3 Multi-key sorting `LeaderboardRules.rank`

DEFINITION

Sorting by a primary key and breaking ties with secondary keys, using a stable comparison sort (O(n log n)).

WHY IT MATTERS HERE

A leaderboard must give everyone a clear, fair position, even when two players have the same points.

HOW WE APPLIED IT

```
rows.sortedWith(
  compareByDescending { it.points }      // 1. most XP
    .thenByDescending { it.correctCount } // 2. more dishes
solved
    .thenBy { it.name.lowercase() })      // 3. A-Z, so ties
are stable
```

IN PLAIN WORDS

Rank by score; if tied, whoever solved more dishes goes higher; if still tied, alphabetical.

INSIGHTS

The final alphabetical key means the order never "flickers" between refreshes for tied players, which players would otherwise read as a bug.

3.4 Threshold lookup & gated progression

`ChefRank · UnlockManager`

DEFINITION

Mapping a number onto ordered ranges (thresholds), and a **state machine** that only allows the next step when the current one is finished.

WHY IT MATTERS HERE

Ranks and level unlocking are the progression that motivates players (objective 2).

HOW WE APPLIED IT

- `ChefRank`: Kusinero 0 → Line Cook 5 → Sous Chef 10 → Head Chef 20 dishes → Kusina Master when every visible dish is solved. The steps match the round badges, so a rank and its badge arrive together.
- `UnlockManager.unlockNextLevel()` unlocks only level N+1, and only if the level just won is the current highest N.

IN PLAIN WORDS

Like a cooking school: you move from apprentice to head chef as you master more dishes, and each class opens only after you pass the one before.

INSIGHTS

Ranks count **dishes solved, not XP**, so a rank reflects how much of the cuisine you've learned, not how fast you type. "Kusina Master" is tied to the total of visible dishes, so it adjusts automatically when admins add levels.

3.5 String normalisation & matching `IngredientArt.forName`

DEFINITION

Converting text to one canonical form (lowercase, letters and digits only) before comparing, often with regular expressions, plus a **fallback chain** when the first lookup fails.

WHY IT MATTERS HERE

Ingredient names arrive written in different ways: "White vinegar", "WHITE VINEGAR", "Vinegar (White/Cane/Coconut)" from the web console.

HOW WE APPLIED IT

```
key = name.lowercase().replace(Regex("[^a-z0-9]+"), "
").trim()
1. look up key in the picture map
2. else find the ingredient with that name in the catalogue →
its id
3. else try "ing_" + key as an id
→ pantry picture for that id, or null (drawn icon)
```

IN PLAIN WORDS

Ignore capitals and punctuation, then try three ways to recognise the ingredient before giving up and drawing a simple icon.

INSIGHTS

Before 1.23, 55 of the web's 123 ingredient names showed a plain icon because only step 1 existed. The test `every ingredient in the book has a picture by its name` now guards all 123.

3.6 Hashing & digital signatures SHA-256 · ECDSA

DEFINITION

A **cryptographic hash** (SHA-256) turns any data into a fixed 64-character fingerprint; changing one character changes the whole fingerprint. A **digital signature** (ECDSA) proves who created a message and that it wasn't altered.

WHY IT MATTERS HERE

They are the algorithms that make the reward ledger trustworthy.

HOW WE APPLIED IT

- Each receipt's id is `'0x' . hash('sha256', kk_canonical_json($receipt))`: canonical JSON first, so the same receipt always hashes the same.
- Blocks are signed with `openssl_sign(…, OPENSSL_ALGO_SHA256)` using an elliptic-curve key on **secp256k1** (the curve Ethereum and Polygon use), with prime256v1 as a fallback on servers without it.
- Custodial wallet addresses are derived from the public key's SHA-256 hash.

IN PLAIN WORDS

The hash is a fingerprint of the receipt; the signature is the system's stamp. Change the receipt and the fingerprint won't match; fake the stamp and it won't verify.

INSIGHTS

Canonical JSON matters: the same data with keys in a different order would otherwise produce a different hash and look "tampered". The unit suite (`api/tests/run.php`) checked these primitives: 51 passed, 0 failed.

UNIT 4

Cybersecurity

Security follows the CIA triad: **confidentiality** (only the right people see data), **integrity** (data isn't changed without detection) and **availability** (the system stays usable). Each module below protects one or more of the three.

4.1 Authentication `bcrypt · bearer tokens · Google · Entra`

DEFINITION

Proving a user is who they claim to be, and keeping that proof safe afterwards.

WHY IT MATTERS HERE

Accounts hold KK, badges and progress. A stolen account is stolen rewards.

HOW WE APPLIED IT

- Passwords are stored only as **bcrypt hashes** (`password_hash(…, PASSWORD_DEFAULT)`) and checked with `password_verify`; the plain password is never stored.
- After login the server issues a random token (`random_bytes`), stores only its **hash** in `api_tokens.token_hash`, and expires it after 30 days. Logging out deletes it.
- The phone sends it as `Authorization: Bearer …` (`KtorClient`).
- Players can sign in with Google (verified server-side); admins sign in to the console with Microsoft Entra.

IN PLAIN WORDS

Your password is turned into an unreadable fingerprint before it's saved. After you log in, you get a temporary pass, and the server only keeps a fingerprint of that too.

INSIGHTS

Hashing tokens server-side means even a leaked database can't be used to log in as players. Before the 17 Aug security review, signing out left tokens valid for 30 days (D-25); now logout revokes them.

4.2 Authorization `identity from the token, never the request`

DEFINITION

Deciding what an authenticated user is allowed to do. The server must derive "who you are" from a trusted credential, not from what the client says.

WHY IT MATTERS HERE

Every reward is tied to a user id. If a client could send any `user_id`, it could write progress into someone else's account.

HOW WE APPLIED IT

- Endpoints call `kk_require_user()`, which reads the bearer token and ignores any `user_id` in the body (fix for D-14, then D-17 for the last four endpoints).
- Admin console: every page re-checks the signed-in admin and their allowed sections (`AdminAuth.IsActiveAdmin`), fails closed if unsure, and signs out after 15 minutes idle.
- Players never touch wallet keys: wallets are **custodial**, managed on the server.

IN PLAIN WORDS

The server doesn't believe "I'm player 46". It checks your pass and decides for itself who you are.

INSIGHTS

D-14 was rated **Critical**: "identity was a claim rather than a credential". It's a classic example of broken access control, the top risk on the OWASP Top 10.

4.3 Brute-force & enumeration defence

`lockout · rate limits · generic errors`

DEFINITION

Limiting repeated guesses and avoiding messages that reveal whether an account exists.

WHY IT MATTERS HERE

Login and password-reset codes can be guessed by a script if attempts are unlimited.

HOW WE APPLIED IT

- Every attempt is logged in `auth_attempts`; too many failures in a time window locks that account for a set number of minutes ("Too many attempts. Try again in … minutes").
- Lockout is per account, not per network (D-33): one person's mistakes on campus Wi-Fi don't lock everyone else.
- Reset codes are bcrypt-hashed and never returned in the API (D-20). Sign-up no longer says which emails exist (D-22), and database errors aren't shown to clients (D-23).

IN PLAIN WORDS

Get the password wrong too many times and that account takes a short break. And the app never confirms whether an email is registered.

INSIGHTS

Returning the reset code in the API response (D-20) meant anyone could reset anyone's password: a full account takeover. It was found by a security review, not by a user, which is why reviews are part of testing.

4.4 Data protection `TLS · AES-256 · secrets · privacy`

DEFINITION

Keeping data confidential **in transit** (encryption over the network) and **at rest** (encryption on disk), and keeping secrets out of code.

WHY IT MATTERS HERE

Tokens, emails and wallet keys pass through phones, Wi-Fi networks and a public GitHub repository.

HOW WE APPLIED IT

- **In transit:** HTTPS with Let's Encrypt certificates (certbot on the API VM); the app's `network_security_config` governs its connections.
- **At rest on the phone:** the token lives in `EncryptedSharedPreferences` (AES-256-GCM, key in the Android Keystore), and `allowBackup="false"` keeps it out of device backups (D-24).
- **At rest on the server:** reward payloads use AES-256-GCM; the master key and wallet private keys live in a vault (`lib/vault.php`), "never in source, in SQL".
- **Repo hygiene:** keystore and passwords are git-ignored; Azure resource paths were masked before publishing test data.
- **Privacy:** UAT responses are confidential under the Data Privacy Act of 2012 (RA 10173).

IN PLAIN WORDS

Everything travels in a locked envelope, the phone keeps your pass in a safe, and the server's master keys are never written into the code.

INSIGHTS

GCM mode adds authentication on top of encryption: the test suite checks that "tampering ciphertext fails AES-256-GCM authentication", so altered data is rejected rather than silently decrypted into garbage.

4.5 Injection prevention `prepared statements`

DEFINITION

SQL injection is when user input is treated as part of a database command. **Prepared statements** send the command and the data separately, so input can never become code.

WHY IT MATTERS HERE

Usernames, guesses, report messages and admin form fields all reach the database.

HOW WE APPLIED IT

64 PHP files on the live API use PDO `prepare()` with named parameters, e.g. `$pdo->prepare("DELETE FROM api_tokens WHERE token_hash = :h")`. The guess validator also accepts letters only (`NotLetters`).

IN PLAIN WORDS

The database gets the question and the answer in separate envelopes, so someone typing a command into the name box just gets a strange name, not a deleted table.

INSIGHTS

Injection is a long-standing entry on the OWASP Top 10. Validating on the phone is for user experience; the server's prepared statements are the real defence, because attackers can skip the app entirely.

4.6 Ledger integrity & auditing `hash chain · PoA · hourly check`

DEFINITION

Integrity means data can't be changed without detection. **Proof of Authority (PoA)** is a blockchain model where approved signers (here, the server) create blocks, instead of public mining.

WHY IT MATTERS HERE

KK and badges should not be editable by anyone, admins included. This is the study's core contribution.

HOW WE APPLIED IT

- Each reward is encrypted, hashed and signed (`kk_phase2_assemble_sign_poa`), then appended to the hash-linked chain under a lock so two writes can't collide.
- The admin Audit page lists the whole ledger, and an hourly whole-chain check shows a **Chain integrity** card.
- The phone only ever shows balances and status read from the API: no seed phrases, keys or gas fees (a project non-negotiable).

IN PLAIN WORDS

Every reward is written in ink in a numbered, fingerprinted book. If anyone erases a line, the hourly check spots it.

INSIGHTS

Why PoA instead of a public chain like Ethereum? No gas fees per reward, instant confirmation, and no player wallets to lose, while still keeping the tamper-evidence that the study needs. secp256k1 keys keep the door open to a Polygon-compatible chain later.

Panel might ask: "Can an admin give themselves KK?" They could edit the database, but not the chain; the receipt would be missing or the hashes would break, and the integrity check would flag it.

UNIT 5

Cloud computing

Kusina Kode runs on Microsoft Azure in the `kusina-kode` resource group (Southeast Asia). This unit maps each service to the concept it demonstrates and to the stress-test evidence.

5.1 Cloud service models `IaaS · PaaS · SaaS`

DEFINITION

IaaS rents raw servers you manage yourself. **PaaS** rents a managed platform where the provider runs the server and you supply the app or data. **SaaS** is finished software you just use.

WHY IT MATTERS HERE

Each part of the system has different needs, so it uses a different model.

HOW WE APPLIED IT

PART	AZURE SERVICE	MODEL
PHP REST API + reward chain	Virtual Machine `vm-kusinakode-api`	IaaS
Database (36 tables)	Azure SQL `sql-kusinakode/kusinakode`, Basic tier	PaaS
Admin console	App Service `app-kusinakode-admin2`, B1	PaaS
Uploaded pictures	Blob Storage `stkusinakode/media`	PaaS (storage)
Admin sign-in	Microsoft Entra ID	SaaS identity

IN PLAIN WORDS

We rent a whole kitchen for the API (and run it ourselves), but rent ready-made services for the database, website and photo storage.

INSIGHTS

The API is on a VM because the chain needs a private folder on disk and scheduled jobs (cron), which a full server allows. The database and console are PaaS so Azure handles patches and backups.

5.2 Compute: the API virtual machine `vm-kusinakode-api`

DEFINITION

A virtual machine is a software-defined computer in the provider's data centre, with its own OS, disk and network address.

WHY IT MATTERS HERE

It runs everything the phone talks to: `api.kusinakode.com/kusinakode/REST/`.

HOW WE APPLIED IT

- A Linux VM serving the PHP API from `/var/www/kusinakode`, with the chain in `/var/lib/kusinakode/chain`.
- **Scheduled jobs:** `settle_pending_cron.sh` every 15 minutes (finishes stuck rewards) and a receipt-cache refresh every minute.
- **Operations:** changes are applied with backups first (`*.bak-YYYYMMDD`) through Azure's Run Command.

IN PLAIN WORDS

A computer in Microsoft's building that stays on 24/7 and answers the app's requests.

INSIGHTS

Stress test: at 40 simultaneous players the VM's CPU stayed at or under about 12%. The server is not the bottleneck.

5.3 Managed relational database `Azure SQL · DTU`

DEFINITION

A database service where the provider runs the database engine. Capacity is sold in tiers; Azure's **DTU** (Database Transaction Unit) measures a blend of CPU, memory and disk work.

WHY IT MATTERS HERE

Every level list, leaderboard and reward lookup hits this database.

HOW WE APPLIED IT

Azure SQL Database `kusinakode` on the **Basic** tier, reached by the API's PHP code and Kyla's console. Firewall rules allow only known IPs; administrators sign in with Entra ID or the SQL admin login.

IN PLAIN WORDS

A filing cabinet Microsoft looks after, where every record of the game lives.

INSIGHTS

Stress test: database load rose from 7% (5 players) to **91% peak at 40 players**, while the VM sat near 10%. The database tier is the system's capacity limit. The fix is **vertical scaling** (Basic → Standard), a settings change that costs more per month.

Panel might ask: "How many users can it handle?" Up to 40 at once with zero errors (average response 1.6 s). Past about 50, the Basic database would start queuing; raising its tier is a configuration change.

5.4 Object storage `Blob Storage`

DEFINITION

Storage for files ("blobs") addressed by URL, cheaper and simpler than keeping files on a server's disk.

WHY IT MATTERS HERE

Admins upload dish, equipment and ingredient pictures that the console and app must show.

HOW WE APPLIED IT

Storage account `stkusinakode`, container `media`, folders `levels/` and `equipment/`. Files are readable by direct URL, but the container can't be listed anonymously.

IN PLAIN WORDS

An online photo album: each picture has its own link, but strangers can't browse the album.

INSIGHTS

Keeping pictures out of the database keeps the database small and fast, which matters given that the database is the bottleneck (5.3).

5.5 Identity, domains & certificates `Entra · DNS · TLS`

DEFINITION

Identity as a service handles sign-in centrally. **DNS** maps names like `api.kusinakode.com` to servers, and **TLS certificates** prove a site is genuine and encrypt traffic.

WHY IT MATTERS HERE

Players and admins need memorable, trusted addresses, and admin sign-in needs strong protection.

HOW WE APPLIED IT

- `api.kusinakode.com` → API VM. The app first used a hard-coded campus network IP (D-19), then the Azure VM's IP, and from 1.16 this name.
- `admin.kusinakode.com` → App Service, with Microsoft Entra sign-in for admins.
- Certificates from Let's Encrypt on the VM, renewed automatically by certbot.

IN PLAIN WORDS

The system has proper addresses with a padlock, and admins log in with their Microsoft account.

INSIGHTS

Using a domain name instead of an IP means the server can move or be replaced without releasing a new app; only the DNS record changes.

5.6 Monitoring, performance & scaling

`Azure Monitor · stress test`

DEFINITION

Monitoring collects metrics (CPU, memory, response time). **Stress testing** pushes a system past normal load to find its limits. **Scaling** adds capacity: vertically (a bigger machine or tier) or horizontally (more machines).

WHY IT MATTERS HERE

The study must show the system works for real users, not just on one developer's phone.

HOW WE APPLIED IT

TEST	RESULT
API, 5 → 40 players at once	0 errors · 0.25 s → 1.6 s average
Database load (Azure Monitor)	peak 91% DTU at 40 players
API VM CPU	≤ ~12%
Phone (vivo V17 Pro), 22 min of play	0 crashes · memory flat 146–153 MB
Cold start	0.7 s phone · 2.6 s emulator
Idle server, first request	5.4 s, then ~0.23 s

IN PLAIN WORDS

We pretended 40 people played at once and watched the gauges. Nothing broke; the database was the part working hardest.

INSIGHTS

Measuring, not guessing, found the real limit (the database, not the server) and two real fixes: the 1.26 startup change and the 1.27 image and animation changes. It also showed the Game Map is graphics-bound on a mid-range phone. The full workbook is `docs/testing/stress-mobile-2026-10-05/`.

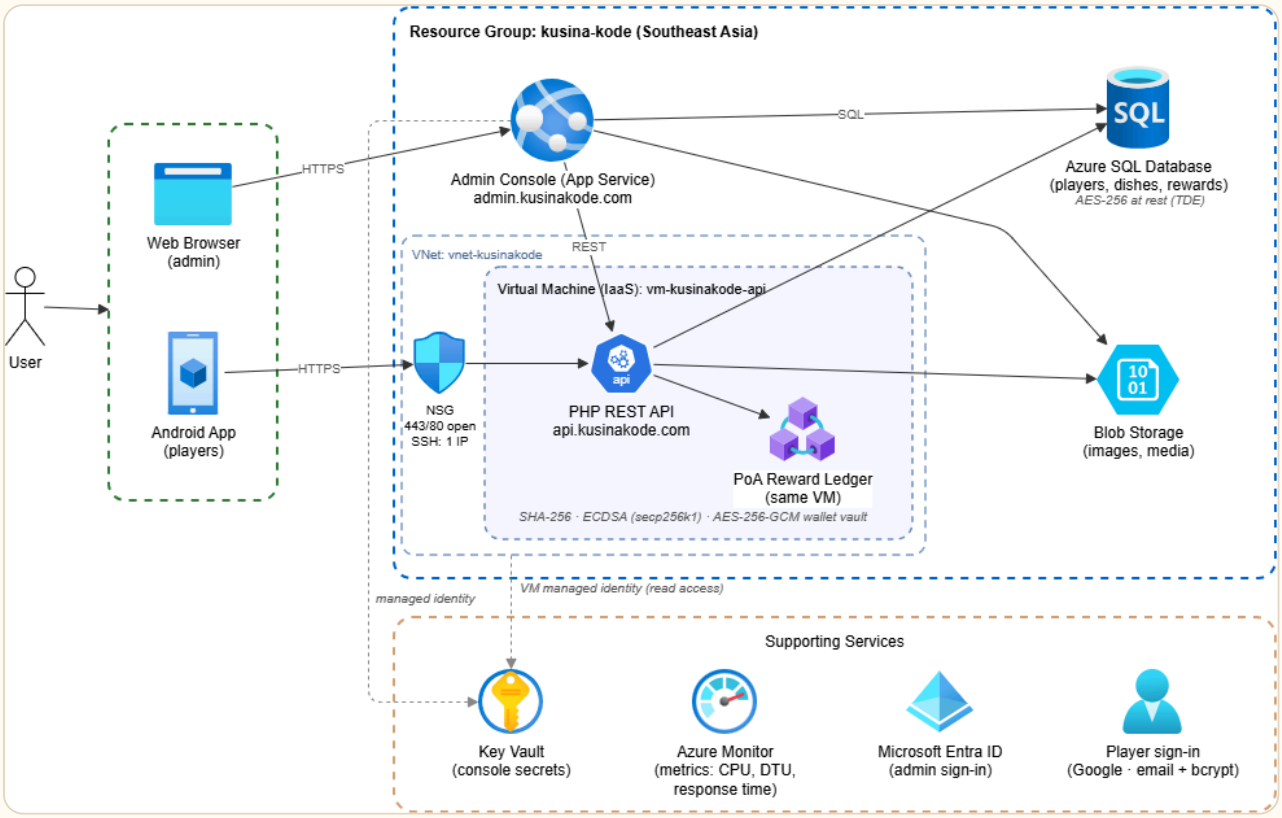
Sources: the Kusina Kode app repository (Kotlin, release 1.27), the KusinaKode-Web repository (ASP.NET + PHP API), the live API on `vm-kusinakode-api` (read-only inspection, 8 Oct 2026), `docs/testing/defects.csv`, and the 5 Oct 2026 stress test workbook.

UNIT 6

System diagrams

Six diagrams the panel will see in the paper, each checked against the live system on 13 Oct 2026. Learn to walk through each one in under a minute: start at the user, follow the arrows, and end at where the data is stored.

6.1 Cloud architecture `Azure · kusina-kode`



Every Azure resource in the `kusina-kode` resource group (Southeast Asia) and how they connect.

WHAT IT SHOWS

Where each part of Kusina Kode runs on Microsoft Azure, and which part talks to which.

HOW TO READ IT

- **Left:** admins use a web browser; players use the Android app. Both connect over **HTTPS**.
- **Admin path:** the **Admin Console** (App Service, `admin.kusinakode.com`) calls the API over REST and also reads Azure SQL and Blob Storage directly.
- **Player path:** the app reaches the **NSG** (firewall: only ports 443/80 open, SSH from one IP), then the **PHP REST API** on the VM (`api.kusinakode.com`) inside `vnet-kusinakode`.
- **Same VM:** the **PoA Reward Ledger** runs next to the API, with its SHA-256 hashes, ECDSA (secp256k1) signatures and AES-256-GCM wallet vault.

- **Right:** data lives in **Azure SQL** (players, dishes, rewards; encrypted at rest with AES-256 TDE) and **Blob Storage** (images, media).
- **Bottom:** supporting services. **Key Vault** holds the console's secrets; both the console and the VM read it through their **managed identities**. **Azure Monitor** tracks CPU, DTU and response time. **Microsoft Entra ID** signs admins in; players sign in with Google or email and a bcrypt-hashed password.

KEY POINTS

It mixes service models: an **IaaS** VM for the API and ledger, and **PaaS** for the console, database and storage. There is no Front Door, Cosmos DB or separate blockchain VM; an earlier draft showed them, and they were removed because they don't exist.

IN PLAIN WORDS

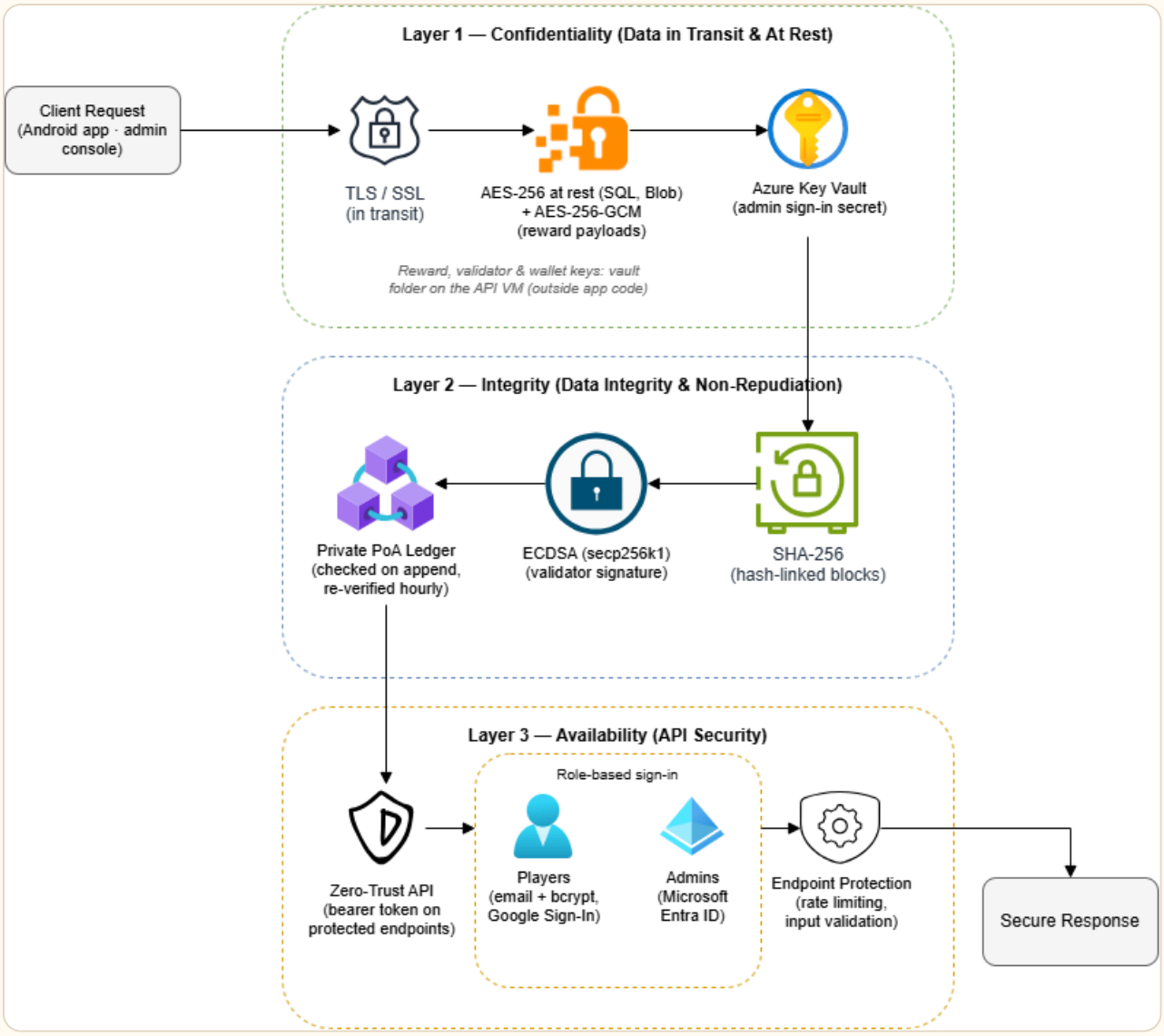
One rented server runs the game's brain and its receipt book. Microsoft looks after the database and the photo album. A guard at the door only lets normal web traffic in, and the passwords sit in a locked safe that only our two services can open.

PANEL MIGHT ASK

Why a VM for the API, not App Service? The ledger keeps files on disk and needs scheduled jobs (finish stuck rewards every 15 minutes, verify the chain every hour), plus a private key folder. A VM gives that control.

Why is the blockchain on the same VM? It's a private proof-of-authority ledger with one validator. A second VM would add cost without adding trust; the hourly integrity check is what proves nothing was changed.

6.2 Cybersecurity architecture `CIA triad`



Every request passes three layers: confidentiality, integrity and availability.

WHAT IT SHOWS The security controls, grouped by the **CIA triad**: **C**onfidentiality (keep data secret), **I**ntegrity (detect any change), **A**vailability (keep the service working for the right people).

HOW TO READ IT

- **Layer 1, confidentiality: TLS** protects data in transit; **AES-256** protects it at rest in SQL and Blob; reward payloads are sealed with **AES-256-GCM**. The admin sign-in secret is in **Azure Key Vault**. The reward, validator and wallet keys sit in a vault folder on the API VM, outside the app code.
- **Layer 2, integrity:** each block carries the **SHA-256** hash of the one before (hash-linked), and the validator signs it with **ECDSA (secp256k1)**. The private PoA ledger checks each block as it is added and re-verifies the whole chain every hour.
- **Layer 3, availability and API security:** a **zero-trust API** (a bearer token is needed on protected endpoints), **role-based sign-in**

(players: email + bcrypt or Google; admins: Microsoft Entra ID), and **endpoint protection** (rate limiting and input validation).

KEY POINTS

The controls are real and testable: login locks after **3** wrong passwords for **15** minutes (HTTP 429); reset codes lock after **5** wrong tries; database queries use prepared statements; the phone stores its token encrypted (AES-256-GCM).

IN PLAIN WORDS

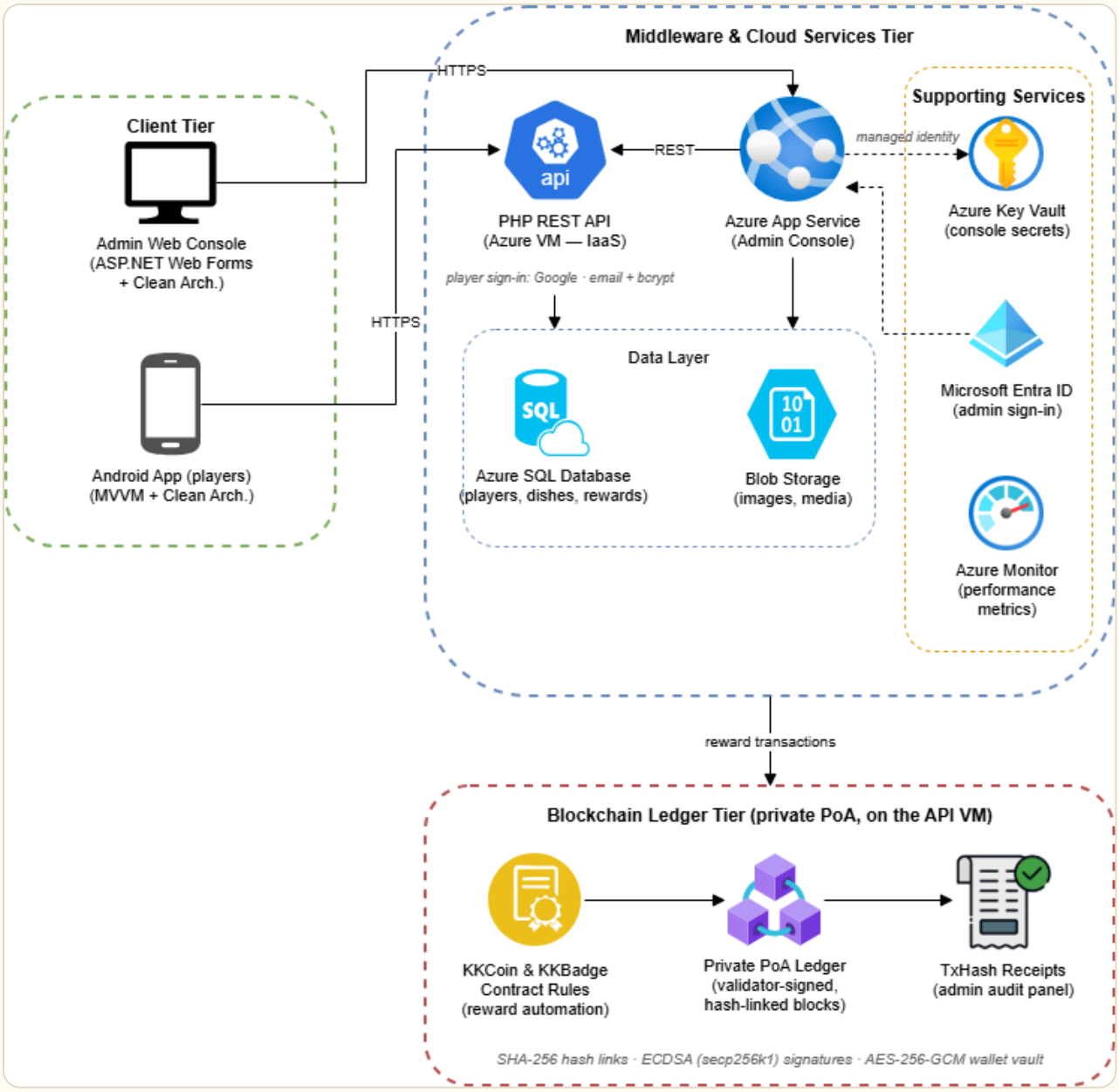
First, everything travels and rests in locked boxes. Second, every reward is written in a numbered, fingerprinted and stamped receipt book, so any change shows. Third, only people with a valid pass get in, and anyone hammering the door is made to wait.

PANEL MIGHT ASK

What if someone edits a reward directly in the database? The ledger's hashes and signatures no longer match, and the hourly check reports it.

Are passwords stored? No. Only bcrypt hashes are stored, and only a hash of each login token.

6.3 System architecture 3 tiers



Client tier, middleware and cloud services tier, and blockchain ledger tier.

WHAT IT SHOWS The system split into **tiers**: layers that each have one job and talk only to the layer next to them.

HOW TO READ IT

- **Client tier:** the **Admin Web Console** (ASP.NET Web Forms, Clean Architecture) and the **Android app** (MVVM, Clean Architecture). Both connect over HTTPS.
- **Middleware and cloud services tier:** the **PHP REST API** (Azure VM) and the **Admin Console** (App Service), the **data layer** (Azure SQL and Blob Storage), and the supporting services (Key Vault, Entra ID, Azure Monitor).
- **Blockchain ledger tier** (private PoA, on the API VM): the **KKCoin and KKBadge contract rules** decide what a reward is worth, the

ledger records it as a signed, hash-linked block, and the **TxHash receipts** appear in the admin audit panel.

KEY POINTS

The arrow labelled **reward transactions** is the only way into the ledger tier: the apps never write to the ledger themselves, they ask the API, and the API applies the contract rules first.

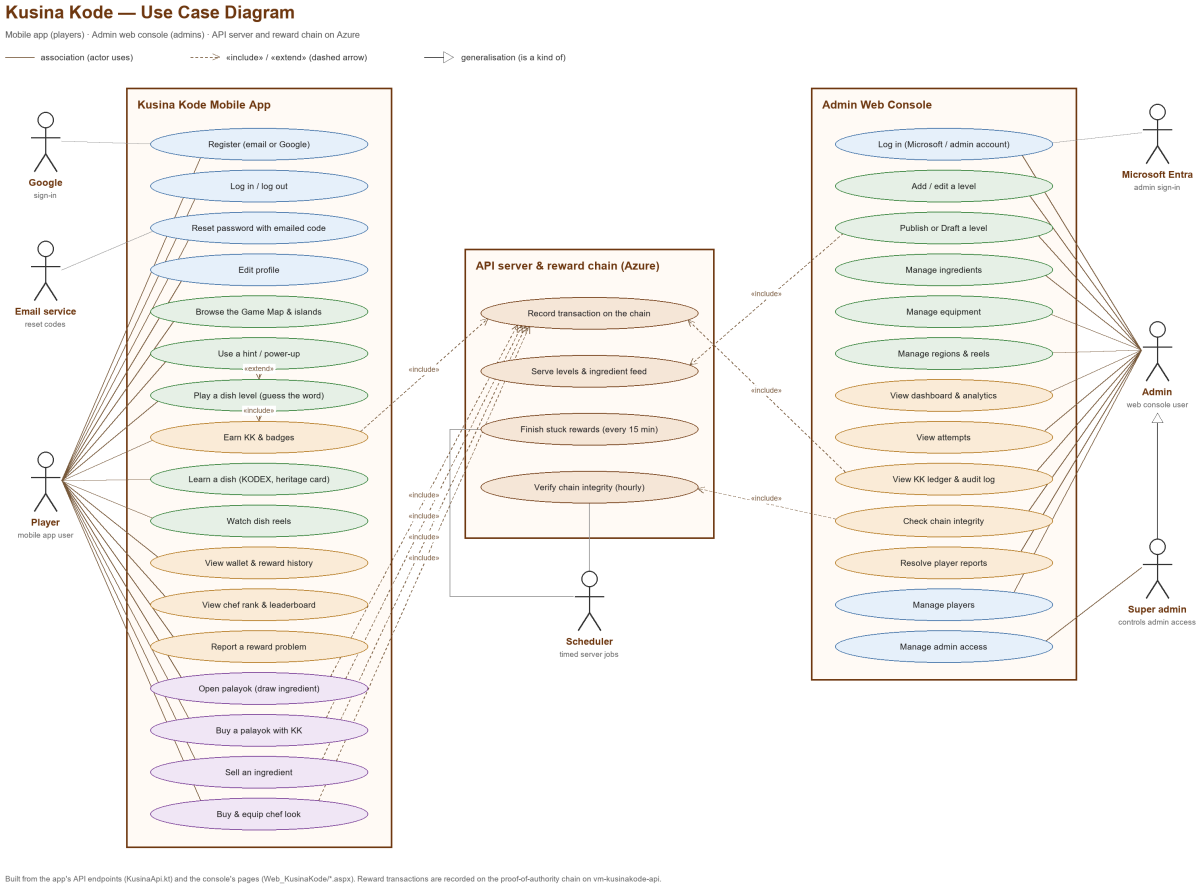
IN PLAIN WORDS

The front counter (apps) takes orders, the kitchen (API and database) prepares them, and the cashier's sealed book (ledger) records every payment. Customers never write in the book themselves.

PANEL MIGHT ASK

Why separate tiers? Each tier can change without breaking the others. For example, the database was normalized on 12–13 Oct without changing the app, because the API kept the same replies.

6.4 Use case diagram UML



Who uses the system and what they can do, across the mobile app, the API and reward chain, and the admin console.

WHAT IT SHOWS

The **actors** (people and outside systems) and the **use cases** (things they can do), drawn inside three system boundaries.

HOW TO READ IT

- **Actors: Player**, **Admin** and **Super admin** (a kind of Admin: the hollow arrow is generalisation), plus **Google**, **Email service**, **Microsoft Entra** and the **Scheduler** (timed server jobs).
- **Solid lines** mean the actor uses that feature. **Dashed arrows** are «include» (always happens as part of it) or «extend» (optional add-on).
- Examples: *Play a dish level* «include» *Earn KK & badges*; *Use a hint / power-up* «extend» *Play a dish level*; every KK action «include» *Record transaction on the chain*.
- The Scheduler runs *Finish stuck rewards* every 15 minutes and *Verify chain integrity* every hour.

KEY POINTS

Note: *Report a reward problem* (player) and *Resolve player reports* (admin) are drawn, but the feature is switched off in the app at the adviser's request. It is kept in the code and database so it can be restored.

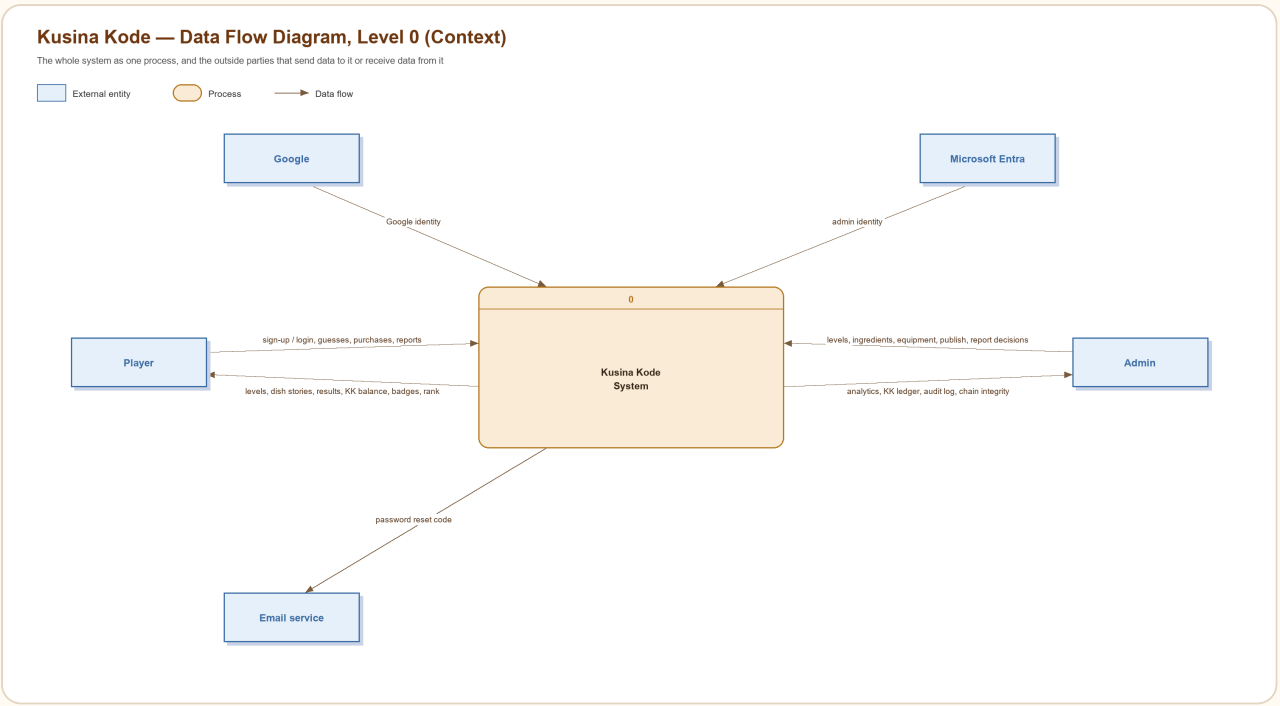
IN PLAIN WORDS

A map of who does what: players play, learn, shop and earn; admins manage the content and check the books; the server's clock does the routine jobs.

PANEL MIGHT ASK

Difference between «include» and «extend»? «Include» always happens (solving a level always earns KK). «Extend» only sometimes happens (a hint is optional while playing).

6.5 Data flow diagram, Level 0 Context



The whole system as one process, and the outside parties that exchange data with it.

WHAT IT SHOWS

A **context diagram**: the system as a single process (0), the **external entities** around it, and the data that flows in and out.

HOW TO READ IT

- **Player** sends sign-up/login, guesses and purchases, and gets back levels, dish stories, results, KK balance, badges and rank.
- **Admin** sends levels, ingredients, equipment and publish decisions, and gets back analytics, the KK ledger, the audit log and chain integrity.
- **Google** and **Microsoft Entra** confirm identities; the **Email service** receives password reset codes to send.

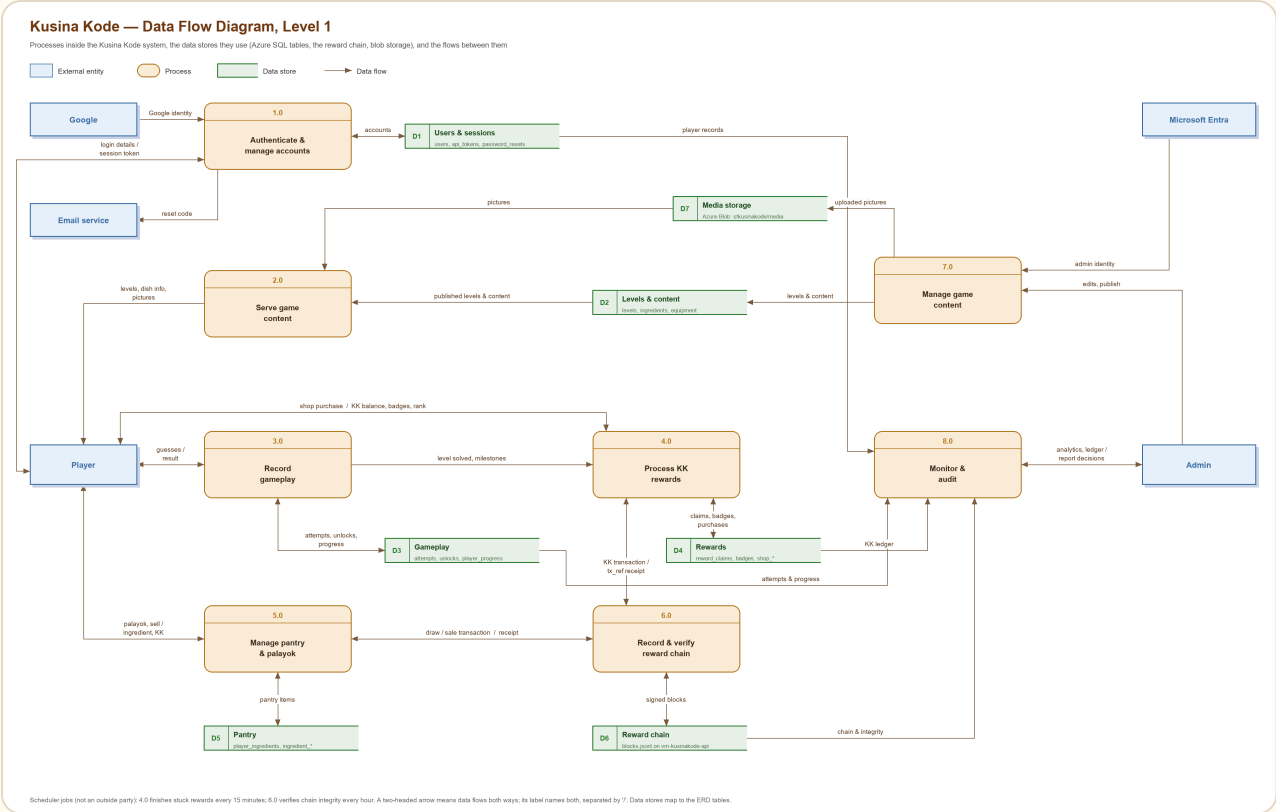
KEY POINTS

The "reports" and "report decisions" flows belong to the reports feature, which is switched off for now (see 6.4).

IN PLAIN WORDS

Treat Kusina Kode as one closed box and list only what goes in and what comes out, and who sends or receives it.

6.6 Data flow diagram, Level 1 Processes & stores



The eight processes inside the system, the seven data stores, and the flows between them.

WHAT IT SHOWS

Level 0's single box opened up: the **processes** (rounded boxes), the **data stores** (green, D1–D7) and the flows between them.

HOW TO READ IT

- **1.0 Authenticate** uses D1 Users & sessions. **2.0 Serve game content** reads D2 Levels & content and D7 Media.
- **3.0 Record gameplay** writes D3 Gameplay and tells **4.0 Process KK rewards** (D4 Rewards) when a level is solved.
- **5.0 Manage pantry & palayok** uses D5 Pantry. Every KK change goes through **6.0 Record & verify reward chain** (D6, signed blocks).
- **7.0 Manage game content** and **8.0 Monitor & audit** are the admin side.
- A two-headed arrow means data flows both ways.

KEY POINTS

The data stores group the database tables by purpose. Since the 12–13 Oct normalization, D2 also covers the new detail tables (for example `level_ingredients`, `level_steps`, `places`) and D4 covers `badge_definitions` and `shop_items`; the flows are unchanged.

IN PLAIN WORDS

Inside the box: who does each job, which drawer each job reads from or writes to, and what gets handed from one job to

the next.

PANEL MIGHT ASK	**Where is the Scheduler?** It's not an outside party, so it isn't drawn as an entity: it's two timed jobs inside 4.0 (every 15 minutes) and 6.0 (every hour), noted under the diagram.

Diagrams: cloud, cybersecurity and system architecture (draw.io, 13 Oct 2026); use case and data flow diagrams (30 Sep 2026). Checked against the Azure resources and the live database on 13 Oct 2026.